

- DTN Phase B Offline Notes
 - 1. Fast Contest Strategy
 - 2. Problem -> First Baseline
 - 3. Data Processing
 - 3.1 Load and inspect fast
 - 3.2 Train/validation split
 - 3.3 Missing values
 - 3.4 Encoding and scaling
 - 3.5 Metrics
 - 4. Classical Machine Learning
 - 4.1 Linear Regression / Ridge / Lasso
 - 4.2 Logistic Regression
 - 4.3 Decision Tree
 - 4.4 XGBoost
 - 4.5 KMeans
 - 4.6 SVM
 - 5. PyTorch Simple Models
 - 5.1 Tabular MLP for classification with nn.Module
 - 5.2 Tabular MLP for regression with nn.Module
 - 5.3 Training function from comp/train_func.py
 - 5.4 How to call that training function
 - 6. PyTorch Optimization Notes
 - 6.1 Correct output / target / loss combinations
 - 6.2 High-value rules
 - 6.3 Tiny-batch overfit test
 - 6.4 DataLoader performance
 - 7. PyTorch Computer Vision
 - 7.1 Preprocessing
 - 7.2 Transfer learning baseline
 - 7.3 Simple CNN with nn.Module
 - 7.4 Vision reminders
 - 8. Embeddings and Text
 - 8.1 Embeddings already given
 - 8.2 Cosine similarity retrieval
 - 8.3 TF-IDF + LogisticRegression
 - 8.4 nn.Embedding
 - 8.5 Offline warning

- [9. Optimization / Solver Problems](#)
 - [9.1 PuLP assignment example](#)
 - [9.2 OR-Tools CP-SAT skeleton](#)
- [10. Debugging Checklist](#)
 - [Common failures](#)
 - [Mandatory optimizer order](#)
 - [Useful prints](#)
- [11. Highest-Value Local Repo Files](#)
- [12. Final Reminder](#)

DTN Phase B Offline Notes

This file is meant to be useful during the contest, not descriptive about the contest.

Keep only what helps you solve problems fast:

- no normal internet
- local docs may exist, but search may be weak
- code templates matter more than theory paragraphs
- baseline first, then improvements

1. Fast Contest Strategy

For each problem, identify these immediately:

- task type: regression / classification / clustering / retrieval / optimization
- input type: tabular / text / embeddings / image / sequence
- metric: accuracy / F1 / MAE / RMSE / custom
- output format: labels / probabilities / CSV / notebook output
- restrictions: fixed model / no downloads / local weights only

Work in this order:

1. understand metric and data
2. build the cheapest correct baseline
3. validate locally
4. submit early
5. improve only if there is clear signal

2. Problem -> First Baseline

Problem pattern	First baseline
tabular classification	XGBoost or LogisticRegression
tabular regression	XGBoost, Ridge, LinearRegression
embeddings given	treat embeddings as normal dense features
text classification	TF-IDF + LogisticRegression
image classification, small dataset	pretrained resnet18 with frozen backbone
custom small neural net task	simple MLP + Adam/AdamW + correct loss
unlabeled vectors	KMeans + PCA visualization
margin-friendly smaller classification dataset	SVM after scaling
assignment / scheduling / constraints	ortools / pulp / z3

3. Data Processing

3.1 Load and inspect fast

```
import pandas as pd
import numpy as np

# Load the training table.
df = pd.read_csv("train.csv")

# Basic shape: rows, columns.
print(df.shape)

# First few rows to inspect column names and obvious issues.
print(df.head())

# Dtypes help you separate numeric vs categorical columns.
print(df.dtypes)
```

```
# Missing values report: start with the worst columns.  
print(df.isna().sum().sort_values(ascending=False).head(20))
```

3.2 Train/validation split

```
from sklearn.model_selection import train_test_split  
  
# Separate features from target.  
X = df.drop(columns=["target"])  
y = df["target"]  
  
# Stratify for classification when the target is class-like.  
# This keeps class ratios similar in train and validation.  
X_train, X_val, y_train, y_val = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y if y.nunique() < 20 else None  
)
```

3.3 Missing values

```
# Numeric columns: median is a safe cheap default.  
num_cols = X_train.select_dtypes(include=["number"]).columns  
  
# Non-numeric columns: usually categorical or text-like.  
cat_cols = X_train.select_dtypes(exclude=["number"]).columns  
  
for c in num_cols:  
    med = X_train[c].median()  
    X_train[c] = X_train[c].fillna(med)  
    X_val[c] = X_val[c].fillna(med)  
  
for c in cat_cols:  
    # Use a visible placeholder instead of dropping rows.  
    X_train[c] = X_train[c].fillna("missing")  
    X_val[c] = X_val[c].fillna("missing")
```

3.4 Encoding and scaling

Rules:

- scale for linear models, SVM, KMeans, neural nets
- do not bother scaling for trees and boosting
- one-hot is fine for linear models and neural nets
- for text use TF-IDF, not raw strings

```
from sklearn.preprocessing import StandardScaler

# Fit only on train data to avoid leakage.
scaler = StandardScaler()

# Scale numeric features only.
Xtr_num = scaler.fit_transform(X_train[num_cols])
Xva_num = scaler.transform(X_val[num_cols])
```

```
# One-hot encode categoricals when using linear models / MLPs.
X_train_oh = pd.get_dummies(X_train, dtype=float)
X_val_oh = pd.get_dummies(X_val, dtype=float)

# Align columns because validation may miss categories seen in train or vice versa.
X_val_oh = X_val_oh.reindex(columns=X_train_oh.columns, fill_value=0)
```

3.5 Metrics

```
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    mean_absolute_error,
    mean_squared_error,
    confusion_matrix,
    classification_report,
)
```

4. Classical Machine Learning

4.1 Linear Regression / Ridge / Lasso

Use when the target is continuous.

```

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Ridge is often safer than plain LinearRegression because it regularizes weights.
model = Ridge(alpha=1.0)

# Train on the scaled numeric matrix.
model.fit(Xtr_num, y_train)

# Predict on validation data.
pred = model.predict(Xva_num)

# MAE is robust and easy to interpret.
mae = mean_absolute_error(y_val, pred)

# RMSE punishes large mistakes more strongly.
rmse = mean_squared_error(y_val, pred, squared=False)

print("MAE:", mae)
print("RMSE:", rmse)

```

4.2 Logistic Regression

Use for classification, especially tabular data or TF-IDF features.

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

# class_weight="balanced" is often useful when classes are imbalanced.
model = LogisticRegression(
    max_iter=2000,
    n_jobs=-1,
    class_weight="balanced"
)

# Train on scaled or sparse features.
model.fit(Xtr_num, y_train)

# Class predictions.
pred = model.predict(Xva_num)

print("acc:", accuracy_score(y_val, pred))
print("f1:", f1_score(y_val, pred, average="weighted"))

```

4.3 Decision Tree

Good cheap baseline. Easy to overfit.

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

# Keep depth limited at first.
model = DecisionTreeClassifier(
    max_depth=6,
    min_samples_leaf=5,
    random_state=42
)

# Tree models can use the original tabular data directly.
model.fit(X_train, y_train)
pred = model.predict(X_val)
```

Notes:

- increase `max_depth` only if clearly underfitting
- use `min_samples_leaf` to reduce noisy splits
- trees do not need scaling

4.4 XGBoost

Very strong default for tabular problems.

```
from xgboost import XGBClassifier, XGBRegressor

# Strong default classifier for many tabular tasks.
model = XGBClassifier(
    n_estimators=400,          # more trees, smaller steps
    max_depth=6,              # not too deep
    learning_rate=0.05,       # safer than 0.3
    subsample=0.9,            # mild row sampling
    colsample_bytree=0.9,     # mild feature sampling
    eval_metric="mlogloss",   # avoid warning + set classification metric
    random_state=42
)

# X_train_encoded should be numeric only.
model.fit(X_train_encoded, y_train)
pred = model.predict(X_val_encoded)
```

```
# Strong default regressor for tabular regression.
reg = XGBRegressor(
    n_estimators=500,
```

```

max_depth=6,
learning_rate=0.05,
subsample=0.9,
colsample_bytree=0.9,
random_state=42
)

reg.fit(X_train_encoded, y_train)
pred = reg.predict(X_val_encoded)

```

4.5 KMeans

Use for unsupervised clustering.

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# KMeans is distance-based, so scaling is important.
X_scaled = scaler.fit_transform(X)

best_k = None
best_score = -1

for k in range(2, 11):
    # n_init=20 gives more stable clustering.
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
    labels = km.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)

    if score > best_score:
        best_score = score
        best_k = k

print("best_k:", best_k, "silhouette:", best_score)

# PCA is a fast way to visualize high-dimensional clusters.
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)

```

4.6 SVM

Good for smaller datasets. Do not start here if the dataset is huge.

```

from sklearn.svm import SVC, SVR

```



```
# RBF kernel is a common nonlinear baseline.
clf = SVC(
    C=1.0,
    kernel="rbf",
    gamma="scale",
    class_weight="balanced"
)

# SVM needs scaled features.
clf.fit(Xtr_num, y_train)
pred = clf.predict(Xva_num)
```

5. PyTorch Simple Models

Use the exact training function style you already use.

5.1 Tabular MLP for classification with `nn.Module`

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# Pick GPU if available.
device = "cuda" if torch.cuda.is_available() else "cpu"

# Convert NumPy arrays to PyTorch tensors.
Xtr = torch.tensor(X_train_np, dtype=torch.float32)
Xva = torch.tensor(X_val_np, dtype=torch.float32)
ytr = torch.tensor(y_train_np, dtype=torch.long) # long for CrossEntropyLoss
yva = torch.tensor(y_val_np, dtype=torch.long)

# Build datasets and loaders.
train_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=128, shuffle=True)
val_loader = DataLoader(TensorDataset(Xva, yva), batch_size=256, shuffle=False)

class TabularClassifier(nn.Module):
    def __init__(self, input_dim, num_classes):
        super(TabularClassifier, self).__init__()
        # First hidden layer: expand feature space.
        self.fc1 = nn.Linear(input_dim, 256)
        # Second hidden layer: compress toward useful representation.
        self.fc2 = nn.Linear(256, 128)
        # Final layer outputs raw logits, one per class.
        self.fc3 = nn.Linear(128, num_classes)
        # ReLU is the standard safe activation.
        self.relu = nn.ReLU()
        # Dropout helps fight overfitting.
```

```

        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        # Hidden block 1.
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)

        # Hidden block 2.
        x = self.fc2(x)
        x = self.relu(x)
        x = self.dropout(x)

        # Final logits. No softmax here because CrossEntropyLoss already handles
it.
        x = self.fc3(x)
        return x

model = TabularClassifier(input_dim=Xtr.shape[1],
num_classes=num_classes).to(device)

# Standard classification loss.
criterion = nn.CrossEntropyLoss()

# AdamW is a strong default optimizer.
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

```

5.2 Tabular MLP for regression with `nn.Module`

```

class TabularRegressor(nn.Module):
    def __init__(self, input_dim):
        super(TabularRegressor, self).__init__()
        # Same structure as classification, but final output is a single number.
        self.fc1 = nn.Linear(input_dim, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 1)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)

        x = self.fc2(x)
        x = self.relu(x)

        # One scalar per sample.
        x = self.fc3(x)
        return x

```

```

model = TabularRegressor(input_dim=Xtr.shape[1]).to(device)

# Regression target should be float, not long.
ytr = torch.tensor(y_train_np, dtype=torch.float32)
yva = torch.tensor(y_val_np, dtype=torch.float32)

train_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=128, shuffle=True)
val_loader = DataLoader(TensorDataset(Xva, yva), batch_size=256, shuffle=False)

# MSE is the default regression loss.
criterion = nn.MSELoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

```

5.3 Training function from `comp/train_func.py`

```

import torch
import copy
import torch.nn.functional as F
import matplotlib.pyplot as plt

def train(model, train_loader, test_loader, optimizer, epochs, criterion):
    """
    Trains a PyTorch model, evaluates it on a test set, and plots the results.
    """
    # Select GPU if available.
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Move model once to the device.
    model = model.to(device, non_blocking=True)

    # Decay LR by a factor of 0.1 every 7 epochs.
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)

    # Store curves for plotting later.
    train_losses = []
    test_losses = []
    test_accuracies = []

    # Track best validation accuracy and weights.
    best_accuracy = 0
    best_model_wts = copy.deepcopy(model.state_dict())

    # Main training loop.
    for epoch in range(epochs):
        # Put the model in training mode.
        model.train()
        total_loss = 0

        # Training pass over all mini-batches.

```

```

for batch_x, batch_y in train_loader:
    # Move batch to GPU/CPU.
    batch_x = batch_x.to(device, non_blocking=True)
    batch_y = batch_y.to(device, non_blocking=True)

    # Clear old gradients.
    optimizer.zero_grad()

    # Forward pass.
    outputs = model(batch_x)

    # Compute loss.
    loss = criterion(outputs, batch_y)

    # Backward pass.
    loss.backward()

    # Update weights.
    optimizer.step()

    # Accumulate batch loss.
    total_loss += loss.item()

# Step the scheduler once per epoch.
scheduler.step()

# Average train loss for this epoch.
train_losses.append(total_loss / len(train_loader))

# Validation phase.
model.eval()
total_test_loss = 0
correct = 0
total = 0

# Turn off gradients during validation.
with torch.no_grad():
    for batch_x, batch_y in test_loader:
        # Move validation batch to device.
        batch_x = batch_x.to(device)
        batch_y = batch_y.to(device)

        # Forward pass.
        outputs = model(batch_x)

        # Validation loss.
        loss = criterion(outputs, batch_y)
        total_test_loss += loss.item()

        # Predicted class = largest logit.
        _, predicted = torch.max(outputs.data, 1)
        total += batch_y.size(0)
        correct += (predicted == batch_y).sum().item()

# Average validation loss and accuracy.
epoch_loss = total_test_loss / len(test_loader)
epoch_accuracy = correct / total

```

```

test_losses.append(epoch_loss)
test_accuracies.append(epoch_accuracy)

# Save best model.
if epoch_accuracy > best_accuracy:
    best_accuracy = epoch_accuracy
    best_model_wts = copy.deepcopy(model.state_dict())
    best_epoch = epoch + 1

print(
    f"Epoch {epoch+1}/{epochs}, "
    f"Training Loss: {train_losses[-1]}, "
    f"Testing Loss: {test_losses[-1]}, "
    f"Testing Accuracy: {epoch_accuracy:.4f}"
)

# Restore best validation checkpoint.
model.load_state_dict(best_model_wts)
print(f"Loaded the best model from epoch {best_epoch} with Testing Accuracy:
{best_accuracy:.4f}")

# Plot loss and accuracy curves.
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(1, epochs+1), train_losses, label='Training Loss')
plt.plot(range(1, epochs+1), test_losses, label='Testing Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs+1), test_accuracies, label='Testing Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()

```

5.4 How to call that training function

```

# Build model, loss, optimizer first.
model = TabularClassifier(input_dim=Xtr.shape[1],
    num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

# Then train.
train(
    model=model,
    train_loader=train_loader,
    test_loader=val_loader,

```

```
optimizer=optimizer,  
epochs=20,  
criterion=criterion  
)
```

6. PyTorch Optimization Notes

6.1 Correct output / target / loss combinations

Task	Output	Target dtype	Loss
regression	1 float per sample	float32	MSELoss, L1Loss, SmoothL1Loss
multiclass classification	num_classes logits	long	CrossEntropyLoss
binary classification	1 logit	float32 in {0,1}	BCEWithLogitsLoss

6.2 High-value rules

- use AdamW first
- try $lr=1e-3$ for small models
- try $lr=1e-4$ when finetuning pretrained backbones
- if loss explodes, lower LR
- if model does not learn, overfit 8 samples
- save the best checkpoint, not just the last epoch

6.3 Tiny-batch overfit test

If this cannot drive loss very low, your model, labels, or loss setup is wrong.

```
# Pick 8 training examples only.  
tiny_x = Xtr[:8].to(device)  
tiny_y = ytr[:8].to(device)  
  
for step in range(200):  
    # Forward pass.
```

```

pred = model(tiny_x)

# If regression model returns shape (N, 1), remove the last dim.
if pred.ndim == 2 and pred.shape[1] == 1:
    pred = pred.squeeze(-1)

# Compute loss on the tiny batch.
loss = criterion(pred, tiny_y)

# Standard optimization step order.
optimizer.zero_grad()
loss.backward()
optimizer.step()

print("tiny-batch loss:", loss.item())

```

6.4 DataLoader performance

From your local notes:

- `pin_memory=True` helps when you also use `non_blocking=True`
- too many workers can waste RAM
- 4 to 8 workers is often enough

```

from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_ds,
    batch_size=128,          # medium batch size
    shuffle=True,            # shuffle training data
    num_workers=6,           # not too small, not too large
    pin_memory=True,         # helps faster host->GPU copies
    persistent_workers=True  # avoid worker restart every epoch
)

for xb, yb in train_loader:
    # non_blocking works best with pin_memory=True.
    xb = xb.to(device, non_blocking=True)
    yb = yb.to(device, non_blocking=True)

```

7. PyTorch Computer Vision

7.1 Preprocessing

```

from torchvision import transforms as T

train_tf = T.Compose([
    # Random crop introduces small geometric variety.
    T.RandomResizedCrop(224),
    # Horizontal flip is useful for many natural image tasks.
    T.RandomHorizontalFlip(),
    # Mild color jitter helps robustness.
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05),
    # Convert PIL image to tensor in [0, 1].
    T.ToTensor(),
    # ImageNet normalization for pretrained backbones.
    T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

val_tf = T.Compose([
    # Deterministic validation pipeline.
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

```

7.2 Transfer learning baseline

Best default for small image datasets.

```

import torch
import torch.nn as nn
from torchvision import models

# Load pretrained ResNet18 weights.
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# Freeze all backbone weights first.
for p in model.parameters():
    p.requires_grad = False

# Replace the classifier head with one that matches your classes.
model.fc = nn.Linear(model.fc.in_features, num_classes)
model = model.to(device)

# Train only the new head first.
optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3, weight_decay=1e-4)
criterion = nn.CrossEntropyLoss()

```

Later improvements if time allows:

- unfreeze last residual block
- lower LR to **1e-4**
- continue training for a few epochs

7.3 Simple CNN with **nn.Module**

Use when pretrained ImageNet features do not fit the domain or when the task is intentionally small/simple.

```
import torch
import torch.nn as nn

class SmallCNN(nn.Module):
    def __init__(self, n_cls):
        super(SmallCNN, self).__init__()

        # First conv block: learn low-level edges and textures.
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 32, kernel_size=3, padding=1)

        # Second conv block: learn more abstract patterns.
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(64, 64, kernel_size=3, padding=1)

        # Pooling reduces spatial size and computation.
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # Adaptive pooling makes the model robust to final feature-map size.
        self.gap = nn.AdaptiveAvgPool2d(1)

        # Fully connected classifier head.
        self.fc1 = nn.Linear(64, 128)
        self.fc2 = nn.Linear(128, n_cls)

        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        # Block 1.
        x = self.conv1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.pool(x)

        # Block 2.
        x = self.conv3(x)
        x = self.relu(x)
        x = self.conv4(x)
        x = self.relu(x)
        x = self.pool(x)
```

```

# Global average pooling -> shape becomes (N, 64, 1, 1).
x = self.gap(x)

# Flatten to (N, 64).
x = x.view(x.size(0), -1)

# Classifier head.
x = self.fc1(x)
x = self.relu(x)
x = self.dropout(x)
x = self.fc2(x)

# Raw logits out.
return x

```

7.4 Vision reminders

- PIL / NumPy image shape: (H, W, C)
- PyTorch image shape: (C, H, W)
- PyTorch batch shape: (N, C, H, W)
- OpenCV loads color images in **BGR**, not RGB
- for tiny image datasets, augmentation often matters more than a bigger model

8. Embeddings and Text

8.1 Embeddings already given

Treat embeddings like normal dense numeric features.

```

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load embedding matrix and labels.
X = np.load("train_embeddings.npy")
y = np.load("train_labels.npy")

# Train/validation split first.
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Logistic regression is a very strong cheap baseline on embeddings.

```

```
clf = LogisticRegression(max_iter=2000, n_jobs=-1)
clf.fit(X_train, y_train)

pred = clf.predict(X_val)
print("acc:", accuracy_score(y_val, pred))
```

```
from xgboost import XGBRegressor

# For embedding regression tasks, boosting is usually strong.
reg = XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    random_state=42
)

reg.fit(X_train, y_train)
pred = reg.predict(X_val)
```

8.2 Cosine similarity retrieval

```
import numpy as np

def l2norm(x):
    # Normalize rows to unit length so cosine similarity becomes dot product.
    return x / (np.linalg.norm(x, axis=1, keepdims=True) + 1e-9)

# Normalize corpus/document embeddings.
doc_emb = l2norm(doc_emb)

# Normalize query embedding too.
q_emb = l2norm(q_emb.reshape(1, -1))[0]

# Cosine similarities because all vectors are unit-normalized.
sims = doc_emb @ q_emb

# Top indices with highest similarity.
topk = np.argsort(-sims)[:10]
```

8.3 TF-IDF + LogisticRegression

This should usually be your first text baseline.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from scipy.sparse import hstack

# Word n-grams capture normal word patterns.
vec_word = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.95,
    sublinear_tf=True,
    max_features=200_000,
)

# Character n-grams are very useful for noisy text, spelling variation, and
# Greeklish.
vec_char = TfidfVectorizer(
    analyzer="char_wb",
    ngram_range=(3, 5),
    min_df=3,
)

# Fit on training text only.
Xw_tr = vec_word.fit_transform(train_texts)
Xw_va = vec_word.transform(val_texts)

Xc_tr = vec_char.fit_transform(train_texts)
Xc_va = vec_char.transform(val_texts)

# Combine word and character features.
Xtr = hstack([Xw_tr, Xc_tr]).tocsr()
Xva = hstack([Xw_va, Xc_va]).tocsr()

# Logistic regression is a strong sparse-text baseline.
clf = LogisticRegression(
    max_iter=2000,
    class_weight="balanced",
    n_jobs=-1
)

clf.fit(Xtr, y_train)
pred = clf.predict(Xva)

```

8.4 nn.Embedding

```

import torch
import torch.nn as nn

# vocab_size = number of token ids
# embedding_dim = vector size per token
embed = nn.Embedding(num_embeddings=vocab_size, embedding_dim=128)

```

```
# Each integer is a token id.
token_ids = torch.tensor([
    [1, 4, 7],
    [2, 9, 0]
])

# Output shape: (batch, seq_len, embedding_dim)
out = embed(token_ids)
print(out.shape)
```

8.5 Offline warning

- do not assume you can download Hugging Face models
- use pretrained models only if weights are already available locally
- if not sure, use TF-IDF or provided embeddings first

9. Optimization / Solver Problems

If the problem is discrete and constraint-heavy, think solver before ML.

9.1 PuLP assignment example

```
import pulp

N, M = 5, 3
cost = [
    [4, 2, 8],
    [1, 9, 3],
    [5, 4, 6],
    [7, 2, 3],
    [2, 8, 1]
]

# Minimize total assignment cost.
prob = pulp.LpProblem("assignment", pulp.LpMinimize)

# Binary variable x[i][j] = 1 if item i goes to slot j.
x = [[pulp.LpVariable(f"x_{i}_{j}", cat="Binary") for j in range(M)] for i in range(N)]

# Objective.
prob += pulp.lpSum(cost[i][j] * x[i][j] for i in range(N) for j in range(M))

# Each item must be assigned exactly once.
for i in range(N):
```

```

prob += pulp.lpSum(x[i][j] for j in range(M)) == 1

# Each slot has capacity <= 2.
for j in range(M):
    prob += pulp.lpSum(x[i][j] for i in range(N)) <= 2

# Solve silently.
prob.solve(pulp.PULP_CBC_CMD(msg=False))

```

9.2 OR-Tools CP-SAT skeleton

```

from ortools.sat.python import cp_model

# Create a CP-SAT model.
model = cp_model.CpModel()

# Add variables and constraints here.

# Create solver.
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 10

# Solve.
status = solver.Solve(model)

```

10. Debugging Checklist

When the model is broken:

- print shapes
- print dtypes
- print a few raw samples
- inspect target distribution
- verify train/validation split
- verify loss matches task
- verify target dtype
- check for NaNs
- try a trivial baseline
- overfit a tiny batch
- change learning rate by $\times 10$ and $/10$

Common failures

- `ValueError: inconsistent numbers of samples`
- Input contains NaN
- could not convert string to float
- Expected Long but got Float
- Expected all tensors to be on the same device
- CUDA out of memory
- size mismatch

Mandatory optimizer order

```
# Clear old gradients first.
optimizer.zero_grad()

# Compute gradients.
loss.backward()

# Apply one optimization step.
optimizer.step()
```

Useful prints

```
# Check dimensions first.
print(X.shape, y.shape)

# Check object types.
print(type(X), type(y))

# Check dtypes for hidden bugs.
print(X.dtype, y.dtype)

# Inspect a few rows.
print(df.head())

# Inspect class balance.
print(np.unique(y, return_counts=True))
```

11. Highest-Value Local Repo Files

Review these before the contest:

- [challenges/comp/train_func.py](#)
- [train_function.py](#)
- [challenges/mnist1.py](#)
- [challenges/cfar1.py](#)
- [challenges/CFAR1001.py](#)
- [challenges/nlp.py](#)
- [challenges/extracted.py](#)
- [decision_trees.py](#)
- [challenges/comp/training_bug.md](#)
- [challenges/comp/tutor_advice_classification_vs_regression.md](#)

12. Final Reminder

The usual winning pattern is:

- identify the task correctly
- choose the correct cheap baseline
- validate quickly
- submit
- improve only where evidence says it is worth it